

Linear Regression - Practical Implementation

Virtual Labs - Dayalbagh Educational Institute

Option A: Single Variable Regression

Step 1: Importing Libraries

Importing Libraries

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
print("Libraries Imported");
```

Output:

Libraries Imported

Step 2: Loading Dataset

Read and store the salary dataset

```
data = pd.read_csv('Salary_dataset.csv')
print("Dataset reading complete")
```

Output:

Dataset reading complete

Step 3: Data Analysis

Display the first 5 rows of the dataset

```
data.head()
```

Output:

index	YearsExperience	Salary
0	1.2	39344
1	1.4	46206
2	1.6	37732
3	2.1	43526
4	2.3	39892

Display the last 5 rows of the dataset

```
data.tail()
```

Output:

index	YearsExperience	Salary
25	9.1	105583
26	9.6	116970
27	9.7	112636
28	10.4	122392
29	10.6	121873

Summary statistics for numerical columns

```
data.describe()
```

Output:

	YearsExperience	Salary
count	30.000000	30.000000
mean	5.413333	76004.000000
std	2.837888	27414.429785
min	1.200000	37732.000000
25%	3.300000	56721.750000
50%	4.800000	65238.000000
75%	7.800000	100545.750000
max	10.600000	122392.000000

Concise summary of a DataFrame

```
data.info()
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
#   Column          Non-Null Count  Dtype
---  -
0   YearsExperience  30 non-null    float64
1   Salary          30 non-null    int64
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
```

Detect missing values

```
data.isnull().sum()
```

Output:

```
YearsExperience    0
Salary            0
dtype: int64
```

Return a tuple representing the dimensionality of the DataFrame

```
data.shape
```

Output:

```
(30, 2)
```

Step 4: Data Preprocessing

Convert YearsExperience to the nearest lower integer by removing decimal values

```
data['YearsExperience'] = np.floor(data['YearsExperience']).astype(int)
data
```

Output:

Dataset after Preprocessing:

index	YearsExperience	Salary
0	1	39344
1	1	46206
2	1	37732
3	2	43526
4	2	39892
5	3	56643
6	3	60151
7	3	54446
8	3	64446
9	3	57190
10	4	63219
11	4	55795
12	4	56958
13	4	57082
14	4	61112
15	5	67939
16	5	66030
17	5	83089
18	6	81364
19	6	93941
20	6	91739
21	7	98274
22	8	101303
23	8	113813
24	8	109432
25	9	105583
26	9	116970
27	9	112636
28	10	122392
29	10	121873

Step 5: Model Training

Create feature matrix X and target vector Y

```
X = data.drop(['Salary'], axis=1)
print("Feature matrix X created.")
```

Output:

Feature matrix X created.

Display the first 5 rows of the feature matrix X

```
X.head()
```

Output:

```
YearsExperience
0    1
1    1
2    1
3    2
4    2
```

Create target vector Y

```
Y = data['Salary']
print("Target vector Y created.")
```

Output:

```
Target vector Y created.
```

Display the first 5 rows of the target vector Y

```
Y.head()
```

Output:

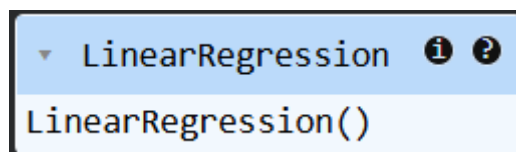
```
0    39344
1    46206
2    37732
3    43526
4    39892
Name: Salary, dtype: int64
```

Split the dataset into training and testing sets

```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size = 0.15, random_state =
    123)
model = LinearRegression()
model.fit(X_train, Y_train)
print("Model Trained")
```

Output:

```
Model Trained
```



Step 6: Model Evaluation

Generate predictions for training and testing datasets using the trained model

```
Y_train_pred = model.predict(X_train)
Y_test_pred = model.predict(X_test)
print("Predictions generated for both Train and Test sets.")
```

Output:

Predictions generated for both Train and Test sets.

Evaluate model performance on training data using MAE, MSE, RMSE, and R2 score

```
mae = mean_absolute_error(Y_train, Y_train_pred)
mse = mean_squared_error(Y_train, Y_train_pred)
rmse = np.sqrt(mse)
r2 = r2_score(Y_train, Y_train_pred)

print("Mean Absolute Error (MAE):", mae)
print("Mean Squared Error (MSE):", mse)
print("Root MSE (RMSE):", rmse)
print("R2 Score:", r2)
```

Output:

Mean Absolute Error (MAE): 5457.139011764704
 Mean Squared Error (MSE): 39230345.16254117
 Root MSE (RMSE): 6263.413219845963
 R2 Score: 0.9422825046716273

Evaluate model performance on testing data using MAE, MSE, RMSE, and R2 score

```
mae = mean_absolute_error(Y_test, Y_test_pred)
mse = mean_squared_error(Y_test, Y_test_pred)
rmse = np.sqrt(mse)
r2 = r2_score(Y_test, Y_test_pred)

print("Mean Absolute Error (MAE):", mae)
print("Mean Squared Error (MSE):", mse)
print("Root MSE (RMSE):", rmse)
print("R2 Score:", r2)
```

Output:

Mean Absolute Error (MAE): 3245.035764705882
 Mean Squared Error (MSE): 20356724.219709344
 Root MSE (RMSE): 4511.842663447978
 R2 Score: 0.9774778470484203

Retrieve the learned coefficient (slope) of the linear regression model

```
model.coef_
```

Output:

```
array([9704.95294118])
```

Retrieve the intercept (bias term) of the linear regression model

```
model.intercept_
```

Output:

```
np.float64(26104.915294117643)
```

Plot actual training data points and the fitted linear regression line for visualization

```
plt.scatter(X_train, Y_train, label='Actual Salary Data')
plt.plot(X_train, Y_train_pred, color='red', label='Fitted Regression Line')
plt.xlabel('Years of Experience'); plt.ylabel('Salary')
plt.title('Linear Regression Fit')
plt.legend(); plt.show()
```

Output:

Linear Regression Fit (Training Data)

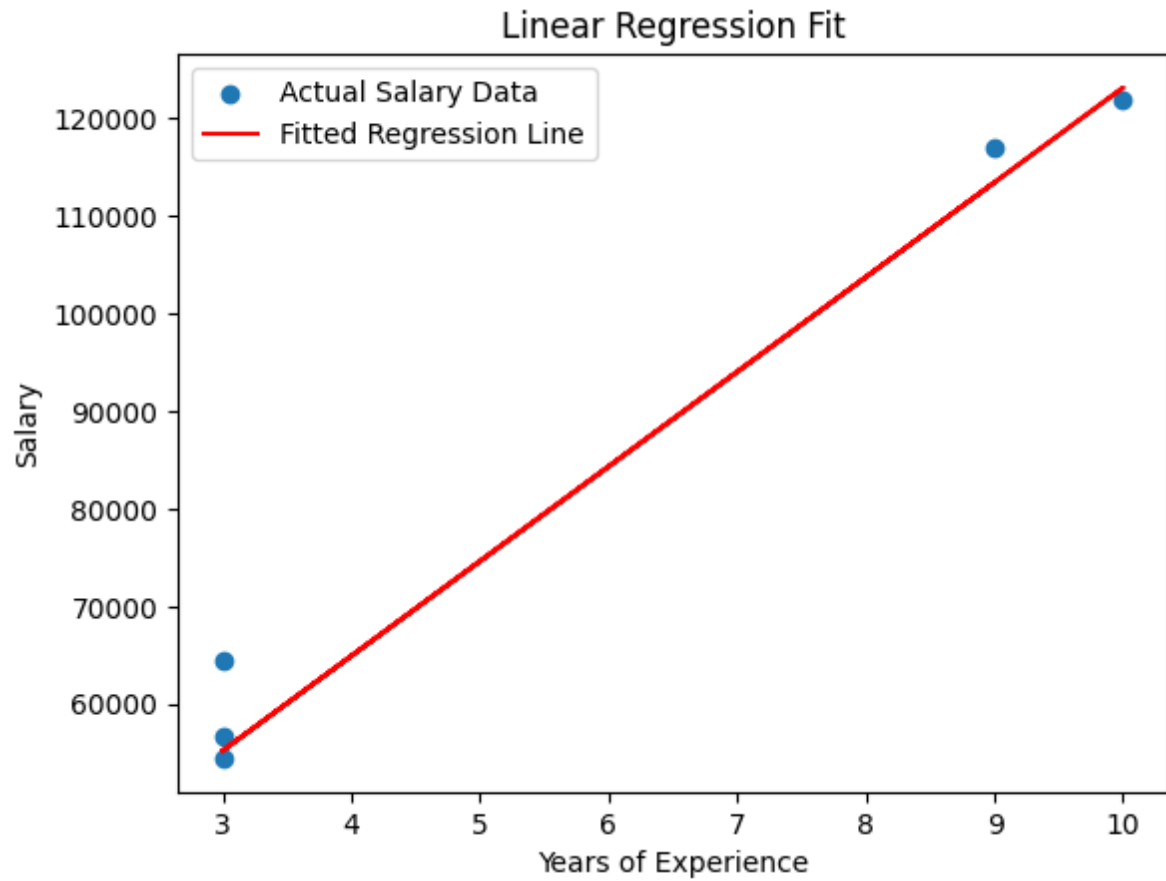


Plot actual testing data points and the fitted linear regression line to evaluate model performance

```
plt.scatter(X_test, Y_test, label='Actual Salary Data')
plt.plot(X_test, Y_test_pred, color='red', label='Fitted Regression Line')
plt.xlabel('Years of Experience'); plt.ylabel('Salary')
plt.title('Linear Regression Fit')
plt.legend(); plt.show()
```

Output:

Linear Regression Fit (Testing Data)

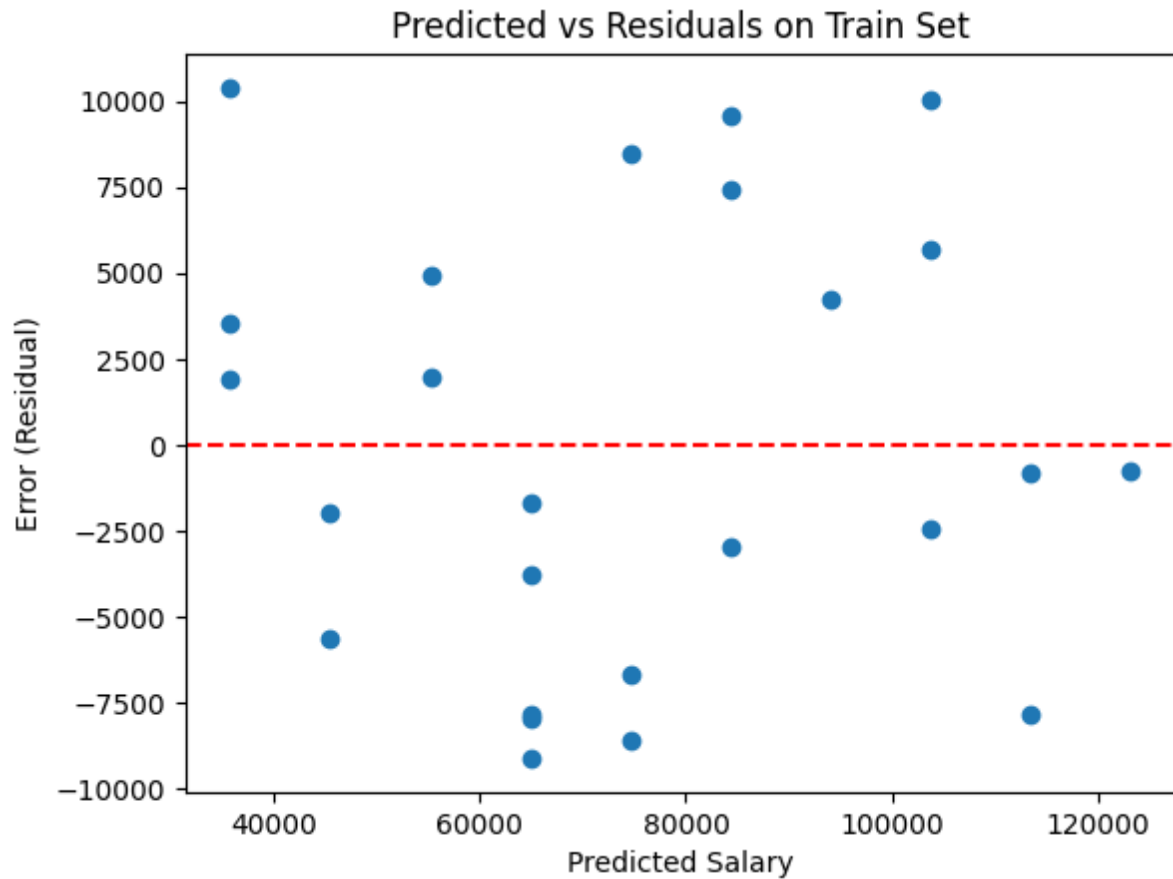


Plot residuals against predicted values on train data to check error patterns and model assumptions

```
residuals = Y_train - Y_train_pred
plt.scatter(Y_train_pred, residuals); plt.axhline(y=0, color='red', linestyle='--')
plt.xlabel("Predicted Salary")
plt.ylabel("Error (Residual)")
plt.title("Predicted vs Residuals on Train Set"); plt.show()
```

Output:

Predicted vs Residuals (Train Set)



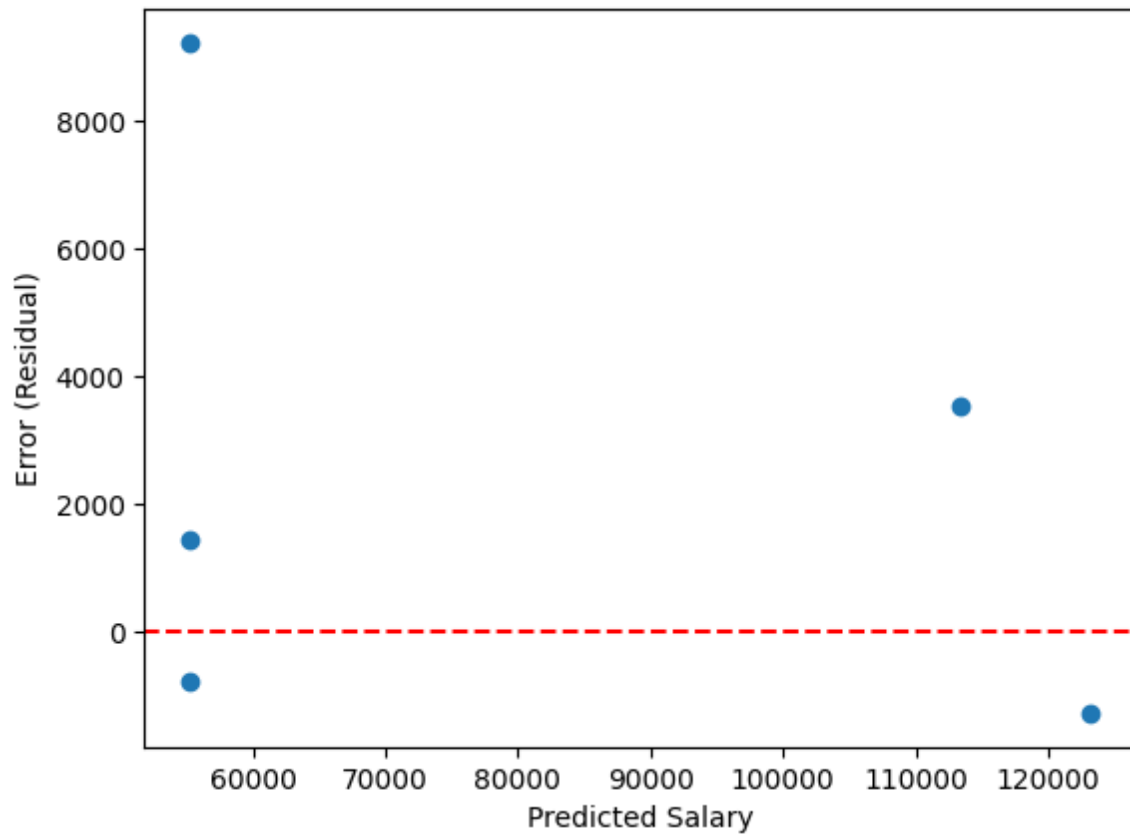
Plot residuals versus predicted values on test data to assess model generalization and error distribution

```
residuals = Y_test - Y_test_pred
plt.scatter(Y_test_pred, residuals)
plt.axhline(y=0, color='red', linestyle='--') # reference line for zero error
plt.xlabel("Predicted Salary")
plt.ylabel("Error (Residual)")
plt.title("Predicted vs Residuals on Test Set")
plt.show()
```

Output:

Predicted vs Residuals (Test Set)

Predicted vs Residuals on Test Set



Predict test values, compare actual vs predicted results, and display the top 5 most accurate predictions

```
Y_test_pred = model.predict(X_test)

comparison_df = pd.DataFrame({
    "Actual Salary": Y_test,
    "Predicted Salary": np.round(Y_test_pred, 2),
    "Error": np.round(Y_test - Y_test_pred, 2),
    "Absolute Error": np.round(abs(Y_test - Y_test_pred), 2)
})

best_predictions = comparison_df.sort_values(by="Absolute Error").head(5)
print(best_predictions)
```

Output:

Best Predictions:

	Actual Selling Pri	Predicted Selling	Error	Absolute Error
7	54446	55219.77	-773.77	773.77
29	121873	123154.44	-1281.44	1281.44
5	56643	55219.77	1423.23	1423.23
26	116970	113449.49	3520.51	3520.51
8	64446	55219.77	9226.23	9226.23

Option B: Multi-Variable Regression

Step 1: Importing Libraries

Importing Libraries

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
print("Libraries Imported")
```

Output:

```
Libraries Imported
```

Step 2: Loading Dataset

Read and store the Car dataset

```
data = pd.read_csv('car_data.csv')
print("Dataset reading complete")
```

Output:

```
Dataset reading complete
```

Step 3: Data Analysis

Display specific selected rows from the dataset using their index positions

```
rows_to_show = [31, 144, 42, 114, 89]
data.loc[rows_to_show]
```

Output:

Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Seller_Type
31 ritz	2011	2.35	4.89	54200	Petrol	Dealer
Manual 0						
144 Bajaj Pulsar NS 20	2014	0.60	0.99	25000	Petrol	Individual
Manual 0						
42 sx4	2008	1.95	7.15	58000	Petrol	Dealer
Manual 0						
114 Royal Enfield Clas	2015	1.15	1.47	17000	Petrol	Individual
Manual 0						
89 etios g	2014	4.75	6.76	40000	Petrol	Dealer
Manual 0						

Display the first 5 rows of the dataset

```
data.head()
```

Output:

	Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Seller_Type	Transmission
0	ritz	2014	3.35	5.59	27000	Petrol	Dealer	Manual
1	sx4	2013	4.75	9.54	43000	Diesel	Dealer	Manual
2	ciaz	2017	7.25	9.85	6900	Petrol	Dealer	Manual
3	wagon r	2011	2.85	4.15	5200	Petrol	Dealer	Manual
4	swift	2014	4.60	6.87	42450	Diesel	Dealer	Manual

Display the last 5 rows of the dataset

```
data.tail()
```

Output:

	Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Seller_Type	Transmission
296	city	2016	9.50	11.6	33988	Diesel	Dealer	Manual
297	brio	2015	4.00	5.9	60000	Petrol	Dealer	Manual
298	city	2009	3.35	11.0	87934	Petrol	Dealer	Manual
299	city	2017	11.50	12.5	9000	Diesel	Dealer	Manual
300	brio	2016	5.30	5.9	5464	Petrol	Dealer	Manual

Show statistical summary of numerical columns

```
data.describe()
```

Output:

	Year	Selling_Price	Present_Price	Kms_Driven	Owner
count	301.000000	301.000000	301.000000	301.000000	301.000000
mean	2013.627907	4.661296	7.628472	36947.205980	0.043189
std	2.891554	5.082812	8.644115	38886.883882	0.247915
min	2003.000000	0.100000	0.320000	500.000000	0.000000
25%	2012.000000	0.900000	1.200000	15000.000000	0.000000
50%	2014.000000	3.600000	6.400000	32000.000000	0.000000
75%	2016.000000	6.000000	9.900000	48767.000000	0.000000
max	2018.000000	35.000000	92.600000	500000.000000	3.000000

Display dataset structure and data types

```
data.info()
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 301 entries, 0 to 300
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
0   Car_Name        301 non-null   object
1   Year            301 non-null   int64
2   Selling_Price   301 non-null   float64
3   Present_Price   301 non-null   float64
4   Kms_Driven      301 non-null   int64
5   Fuel_Type       301 non-null   object
6   Seller_Type     301 non-null   object
7   Transmission    301 non-null   object
8   Owner           301 non-null   int64
dtypes: float64(2), int64(3), object(4)
memory usage: 21.3+ KB
```

Check the number of missing values in each column

```
data.isnull().sum()
```

Output:

```
Column          0
Car_Name        0
Year            0
Selling_Price   0
Present_Price   0
Kms_Driven      0
Fuel_Type       0
Seller_Type     0
Transmission    0
Owner           0
dtype: int64
```

Show the number of rows and columns in the dataset

```
data.shape
```

Output:

```
(301, 9)
```

Count occurrences of each Fuel_Type in the dataset

```
data['Fuel_Type'].value_counts()
```

Output:

```
Fuel_Type  count
Petrol      239
Diesel      60
CNG         2
dtype: int64
```

Count occurrences of each Seller_Type in the dataset

```
data['Seller_Type'].value_counts()
```

Output:

```

Seller_Type  count
Dealer      195
Individual   106
dtype: int64

```

Count occurrences of each Transmission type in the dataset

```
data['Transmission'].value_counts()
```

Output:

```

Transmission  count
Manual        261
Automatic      40
dtype: int64

```

Step 4: Data Preprocessing

Encode Fuel_Type column by mapping categorical values to numerical (Petrol:0, Diesel:1, CNG:2)

```

data = data.replace({'Fuel_Type':{'Petrol':0, 'Diesel':1, 'CNG':2}})
data

```

Output:

Dataset after Fuel_Type Encoding:

	Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Seller_Type	Transmission
Owner								
0	ritz	2014	3.35	5.59	27000	0	Dealer	Manual
0								
1	sx4	2013	4.75	9.54	43000	1	Dealer	Manual
0								
2	ciaz	2017	7.25	9.85	6900	0	Dealer	Manual
0								
3	wagon r	2011	2.85	4.15	5200	0	Dealer	Manual
0								
4	swift	2014	4.60	6.87	42450	1	Dealer	Manual
0								
...	...	...	...	...	...	...	...	...
...								
296	city	2016	9.50	11.60	33988	1	Dealer	Manual
0								
297	brio	2015	4.00	5.90	60000	0	Dealer	Manual
0								
298	city	2009	3.35	11.00	87934	0	Dealer	Manual
0								
299	city	2017	11.50	12.50	9000	1	Dealer	Manual
0								
300	brio	2016	5.30	5.90	5464	0	Dealer	Manual
0								
301 rows x 9 columns								

Encode Seller_Type column by mapping categorical values to numerical (Dealer:0, Individual:1)

```

data = data.replace({'Seller_Type':{'Dealer':0, 'Individual':1}})
data

```

Output:

Dataset after Seller_Type Encoding:

	Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Seller_Type	Transmission
0	ritz	2014	3.35	5.59	27000	0	0	Manual
0								
1	sx4	2013	4.75	9.54	43000	1	0	Manual
0								
2	ciaz	2017	7.25	9.85	6900	0	0	Manual
0								
3	wagon r	2011	2.85	4.15	5200	0	0	Manual
0								
4	swift	2014	4.60	6.87	42450	1	0	Manual
0								
...	...	...	...	...	...	...	...	...
...								
296	city	2016	9.50	11.60	33988	1	0	Manual
0								
297	brio	2015	4.00	5.90	60000	0	0	Manual
0								
298	city	2009	3.35	11.00	87934	0	0	Manual
0								
299	city	2017	11.50	12.50	9000	1	0	Manual
0								
300	brio	2016	5.30	5.90	5464	0	0	Manual
0								

301 rows x 9 columns

Encode Transmission columnn by mapping categorical values to numerical (Manual:0, Automatic:1)

```
data = data.replace({'Transmission':{'Manual':0, 'Automatic':1}})
data
```

Output:

Dataset after Transmission Encoding:

	Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Seller_Type	Transmission
0	ritz	2014	3.35	5.59	27000	0	0	0
0								
1	sx4	2013	4.75	9.54	43000	1	0	0
0								
2	ciaz	2017	7.25	9.85	6900	0	0	0
0								
3	wagon r	2011	2.85	4.15	5200	0	0	0
0								
4	swift	2014	4.60	6.87	42450	1	0	0
0								
...	...	...	...	...	...	...	...	...
...								
296	city	2016	9.50	11.60	33988	1	0	0
0								
297	brio	2015	4.00	5.90	60000	0	0	0
0								
298	city	2009	3.35	11.00	87934	0	0	0
0								
299	city	2017	11.50	12.50	9000	1	0	0
0								
300	brio	2016	5.30	5.90	5464	0	0	0
0								

301 rows x 9 columns

Step 5: Model Training

Create feature matrix X by dropping Car_Name and Selling_Price columns

```
X = data.drop(['Car_Name', 'Selling_Price'], axis=1)
print("Feature matrix X created by dropping 'Car_Name' and 'Selling_Price' columns.")
```

Output:

Feature matrix X created by dropping 'Car_Name' and 'Selling_Price' columns.

Create target vector Y containing the Selling_Price column

```
Y = data['Selling_Price']
print("Target vector Y created from 'Selling_Price' column.")
```

Output:

Target vector Y created from 'Selling_Price' column.

Display the first 5 rows of the feature matrix X

```
X.head()
```

Output:

Feature Matrix (X) Preview:

	Year	Present_Price	Kms_Driven	Fuel_Type	Seller_Type	Transmission	Owner
0	2014	5.59	27000	0	0	0	0
1	2013	9.54	43000	1	0	0	0
2	2017	9.85	6900	0	0	0	0
3	2011	4.15	5200	0	0	0	0
4	2014	6.87	42450	1	0	0	0

Display the first 5 values of the target vector Y

```
Y.head()
```

Output:

Target Vector (Y) Preview:

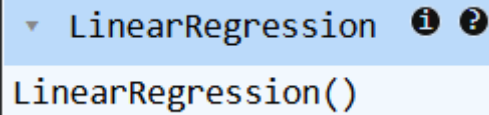
```
Selling_Price
0    3.35
1    4.75
2    7.25
3    2.85
4    4.60
dtype: float64
```

Split the dataset into training and testing sets, then train a Linear Regression model

```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size = 0.15, random_state =
    123)
model = LinearRegression()
model.fit(X_train, Y_train)
print("Model Trained Successfully")
```

Output:

Model Trained Successfully



Step 6: Model Evaluation

Generate predictions for training and testing datasets using the trained model

```
Y_train_pred = model.predict(X_train)
Y_test_pred = model.predict(X_test)
print("Predictions generated for both Train and Test sets.")
```

Output:

Predictions generated for both Train and Test sets.

Evaluate model performance on training data using MAE, MSE, RMSE, and R2 score

```
mae = mean_absolute_error(Y_train, Y_train_pred)
mse = mean_squared_error(Y_train, Y_train_pred)
rmse = np.sqrt(mse)
r2 = r2_score(Y_train, Y_train_pred)

print("Mean Absolute Error (MAE):", mae)
print("Mean Squared Error (MSE):", mse)
print("Root MSE (RMSE):", rmse)
print("R^2 Score:", r2)
```

Output:

Mean Absolute Error (MAE): 1.1605491562123338
Mean Squared Error (MSE): 3.015625977191586
Root MSE (RMSE): 1.7365557800403608
R^2 Score: 0.8869753499147778

Evaluate model performance on testing data using MAE, MSE, RMSE, and R2 score

```
mae = mean_absolute_error(Y_test, Y_test_pred)
mse = mean_squared_error(Y_test, Y_test_pred)
rmse = np.sqrt(mse)
r2 = r2_score(Y_test, Y_test_pred)

print("Mean Absolute Error (MAE):", mae)
print("Mean Squared Error (MSE):", mse)
print("Root MSE (RMSE):", rmse)
print("R^2 Score:", r2)
```

Output:

Mean Absolute Error (MAE): 1.3160017493663516
Mean Squared Error (MSE): 3.8164774430610064
Root MSE (RMSE): 1.953580672684903
R^2 Score: 0.8114116982277362

Retrieve the learned coefficients and intercept of the linear regression model

```
print("Coefficients:", model.coef_)
print("Intercept:", model.intercept_)
```

Output:

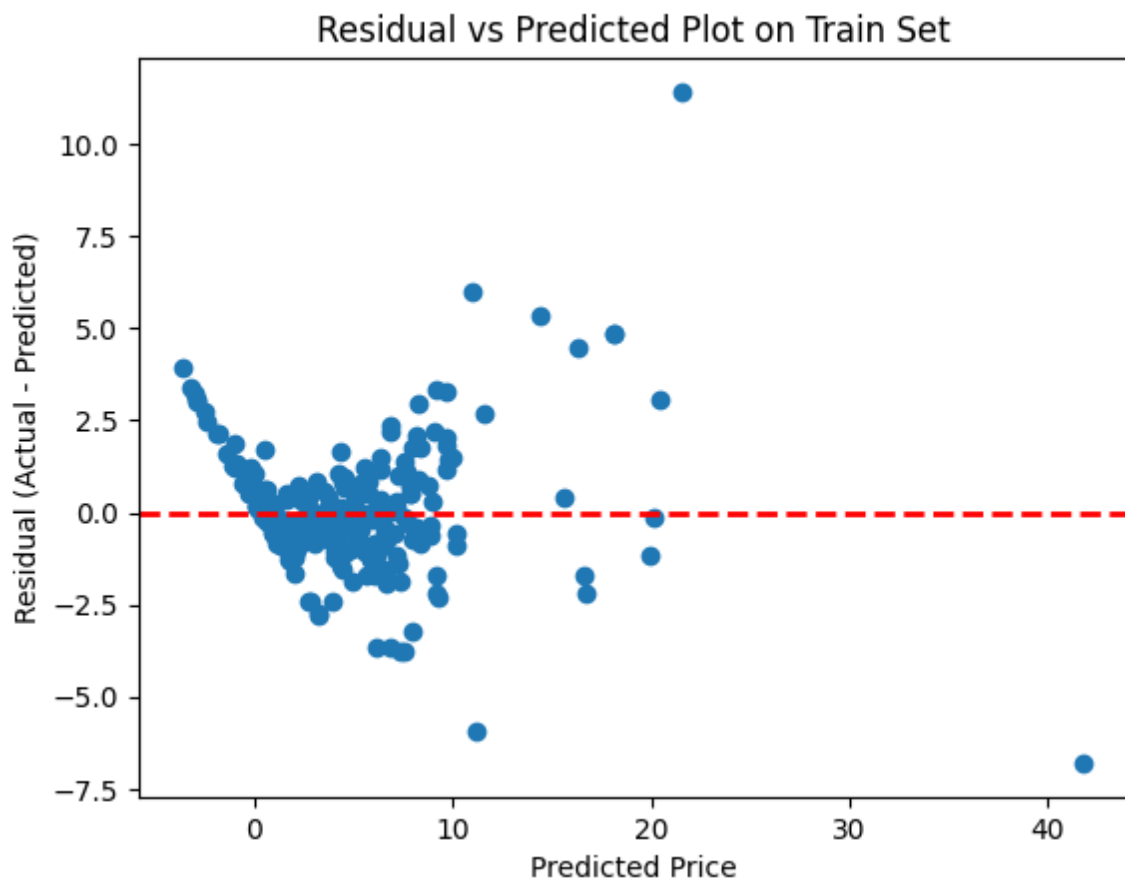
```
Coefficients: [ 3.92825483e-01  4.39836584e-01 -5.25747442e-06  1.38880051e+00 -1.25197058e+00
 1.44799328e+00 -7.41998907e-01]
Intercept: -789.4879877881141
```

Plot residuals against predicted values on training data to analyze error patterns

```
residuals = Y_train - Y_train_pred
plt.scatter(Y_train_pred, residuals)
plt.axhline(0, color='red', linestyle='--', linewidth=2)
plt.xlabel("Predicted Price")
plt.ylabel("Residual (Actual - Predicted)")
plt.title("Residual vs Predicted Plot on Train Set")
plt.show()
```

Output:

```
Residual vs Predicted Plot on Train Set
```

**# Plot residuals against predicted values on testing data to assess model generalization**

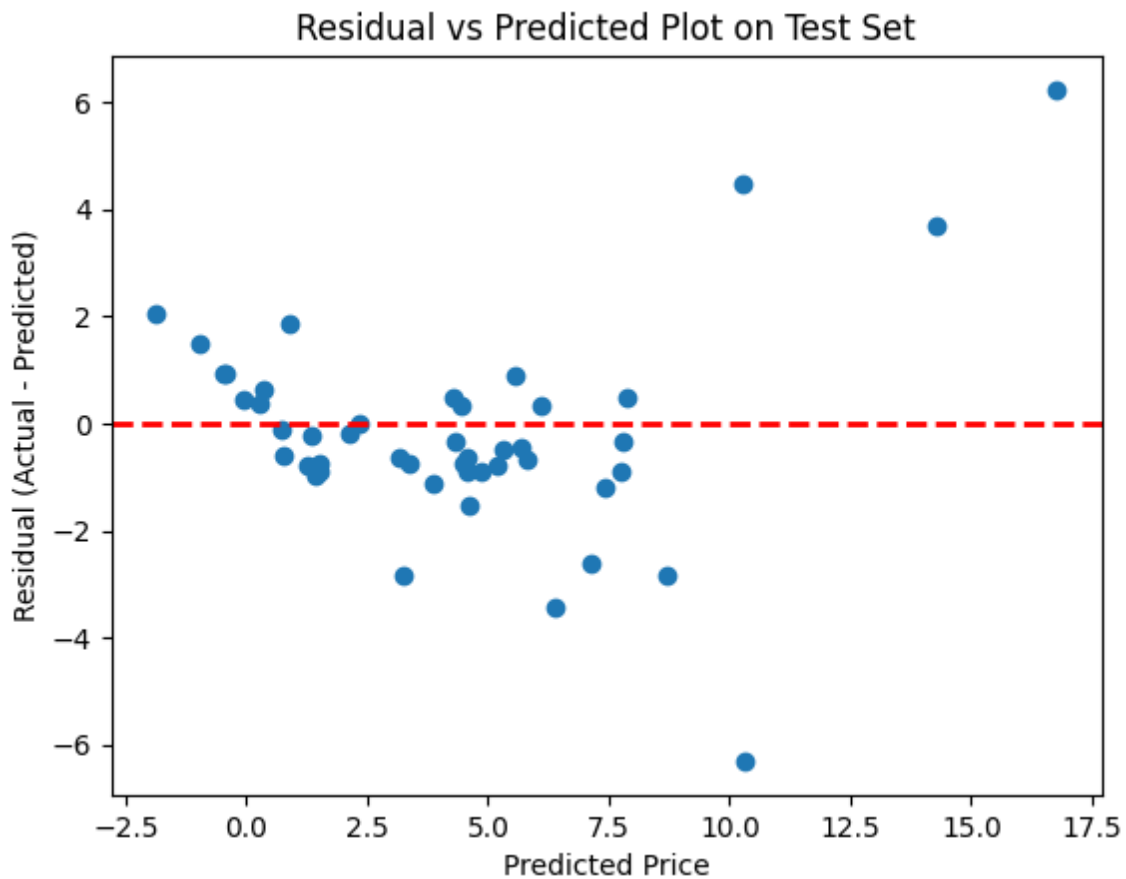
```

residuals = Y_test - Y_test_pred
plt.scatter(Y_test_pred, residuals)
plt.axhline(0, color='red', linestyle='--', linewidth=2)
plt.xlabel("Predicted Price")
plt.ylabel("Residual (Actual - Predicted)")
plt.title("Residual vs Predicted Plot on Test Set")
plt.show()

```

Output:

Residual vs Predicted Plot on Test Set



Choose one random test sample from the dataset to compare the actual and predicted selling prices.

```

test_samples = data.sample(n=5, random_state=42)
display(test_samples)
print("Interactive Prediction Table Loaded")

```

Output:

Interactive Prediction Table Loaded

Choose one random test sample.

Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Seller_Type	Transmission	Owner